Desafio Técnico Meli

Desafio Técnico 2
1. Aplicação “Serviço Anúncios” (obrigatório) 2
Diagrama macro 2
Regras de negócio: 2
Detalhes do funcionamento: 3
2. Estudo de caso (opcional) 4
Informações para entrega do projeto 5
Itens obrigatórios: 5
Itens opcionais / diferencial: 5
Itens que não serão avaliados: 5
Documentação 6
Mockoon via Docker 6
Mockoon aplicativo desktop 6
Documentação das APIs disponíveis com o Mock 6
Resumo das rotas 6
1. Sellers (Tray Meli) 7
1.1 Listar sellers (sem ID) 7
1.2 Seller por ID fixo (252254392) 8
1.3 Seller por ID genérico 8
2. Search Items (Mercado Livre) 9
2.1 Busca de anúncios por seller 9
3. Items (Mercado Livre) 10
3.1 Detalhe de item por ID 10

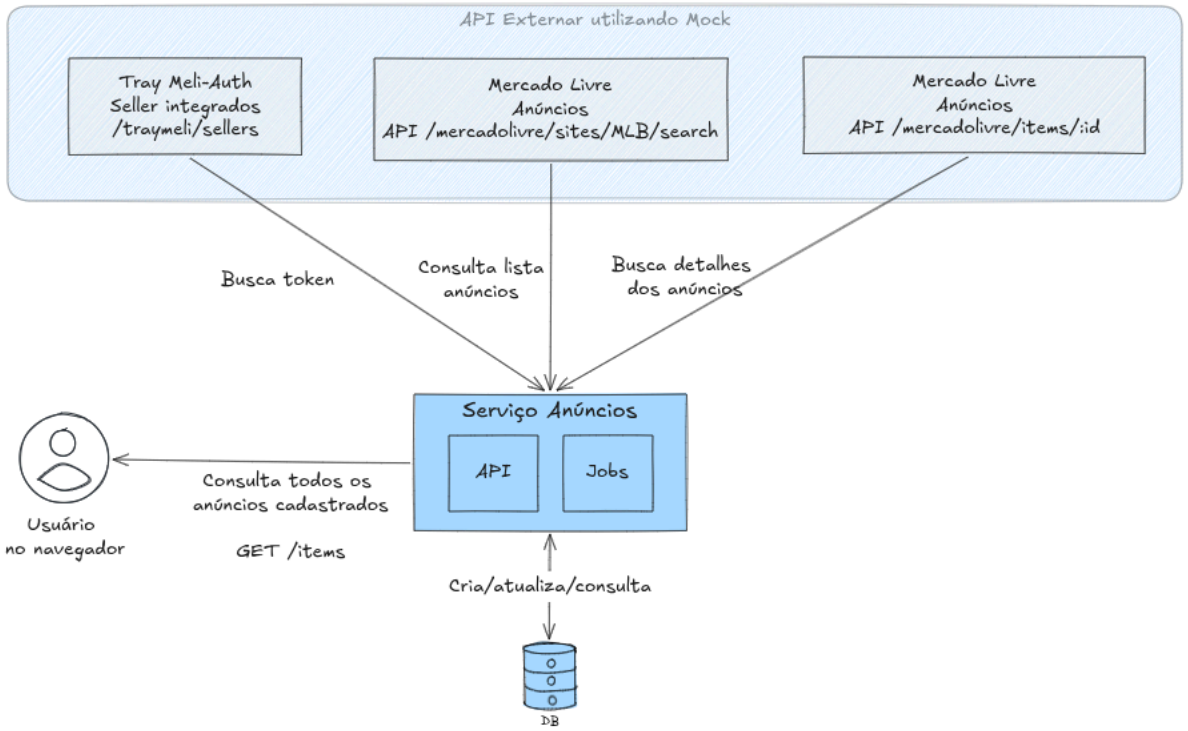
Desafio Técnico

---

1. Aplicação “Serviço Anúncios” (obrigatório)

Desenvolver um serviço que capture todos os anúncios (30) do MercadoLivre e disponibilize os dados destes anúncios em uma API REST em formato JSON utilizando Laravel.

Diagrama macro



Regras de negócio:

1. Para realizar a consultas nas API dos Mercado Livre é obrigatório um access token, para obtê-lo, utilize a API do Meli-Auth com o ID de seller **252254392**;
2. Somente utilize um token válido retornado pelo serviço Mei-Auth [*];
3. Em casos de tokens inválidos, registre um log do problema e prossiga;
4. Consulte primeiramente na API do Mercado Livre Search, utilizando o devido **Bearer token** [*];
5. Com a lista de ids de anúncios, busque no Mercado Livre Items os detalhes de cada anúncios e registre em uma tabela no banco de dados MySQL; Atualize os dados caso execute mais vezes;

6. Demonstre estes anúncios em um API;
7. Apresente os processamentos e logs no terminal;

Sempre que se referir a consultas em APIs de Auth ou Mercado Livre, está se referindo as APIs do mock. Todo projeto deve ser executado localmente.

* Atenção: a API de Auth tenta simular uma situação real, nem todos os retornos são válidos (inactive_token), caso tente usar um token inválido as APIs do Mercado Livre vão retornar erro. A integração precisa estar preparada para estes casos.

Detalhes do funcionamento:

1. Inserir no banco de dados 30 anúncios capturados do Mercado Livre, pode ser através de uma **schedule/job** programada para executar a **cada 10 min**, ou executado manualmente, com as seguintes validações:
    a. Consultar no banco de dados se existem 30 anúncios cadastrados, se não existir realize a consulta de acordo com as regras; Caso já exista, permitir atualizar os dados;
    b. Certifique-se que o job não fique travado, utilize filas para consultar os dados de detalhes e colocá-los no banco de dados.Adicione todos os campos contidos na API de Items e se achar necessário outros campos, como status do processamento.
2. Implementar job para processamento dos anúncios via fila;
3. O sistema deve ter também uma API REST que retorne os dados do banco de dados no formato JSON;

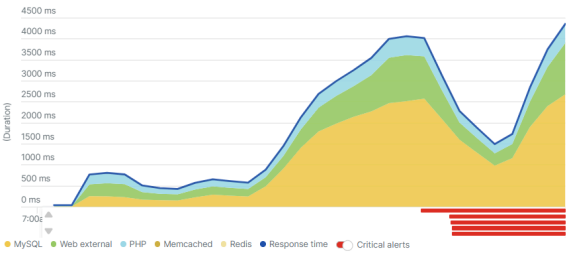
2. Estudo de caso (opcional)

O Serviço Anúncios consulta as APIs do Mercado Livre (Search → Items), persiste os anúncios no MySQL e expõe uma API REST (GET /items) para consulta. O job de captura pode rodar a cada 10 min ou manualmente, usando filas para buscar os detalhes dos itens.

Supondo um cenário em escala: vários sellers com milhares de anúncios.

Considere a evolução do sistema para múltiplos sellers, cada um com milhares de anúncios (ex.: 10 sellers × 5.000 anúncios = 50.000 itens):

Neste cenário é apresentado os seguintes alertas de infraestrutura:

Monitoramentos hardware	Monitoramentos transações
	

Em momentos de pico de acessos e processamento, ocorre:

1. Load average – subida brusca em poucos minutos e permanência elevada, indicando excesso de processos/threads competindo por CPU.
2. Tempo de resposta da aplicação (API /items) – picos de duração de até ~4 s por requisição, com MySQL respondendo como principal componente (maior parte do tempo em operações de banco). Alertas críticos são disparados nesses picos.
3. Fluxo atual – o job busca token (Meli-Auth), depois a lista de IDs (Search) e, em seguida, para cada um dos IDs, enfileira ou dispara chamadas à API Items e grava/atualiza registros no MySQL; em paralelo, a API GET /items atende usuários lendo do mesmo banco.

Proponha uma solução objetiva para resolver este impacto do(s) job(s) no desempenho da aplicação e no tempo de resposta da API. (apresente a solução da forma que achar melhor, descrevendo ou em forma de diagrama auto explicativo)

Informações para entrega do projeto

Itens obrigatórios:

- Disponibilizar o código no Github;

Itens opcionais / diferencial:

- docker-compose do projeto;
- documentar o serviço no readme do projeto de forma que possa ser executado;
- Aplicar padrões de projeto, separação de código para entendimento de todo time. Documenta as decisões;
- Pelo menos um teste unitário;
- Realizar commits parciais, conforme desenvolvimento;
- Estudo de caso opcional.

Itens que não serão avaliados:

- Frontend, não há necessidade e todo conteúdo pode ser exposto na API e no terminal;
- Teste unitário;

Documentação

Para a simulação das APIs do Meli-Auth e do Mercado Livre, utilize o Mockoon (https://mockoon.com).

Mockoon via Docker

- Fazer o download do arquivo mocketplace.json
- Executar um docker container com o arquivo mocketplace.json:
- **`docker run -d --mount type=bind,source=./mocketplace.json,target=/data,readonly -p 3000:3000 mockoon/cli:latest -d data -p 3001`**
- ou utilizar no docker-compose do projeto.

Mockoon aplicativo desktop

- 1. Abra o arquivo `meli-mockoon.json` no Mockoon (File → Open).
- 2. Inicie o servidor (porta 3001).
- 3. Use a base URL `http://localhost:3001` em ferramentas como Postman, Bruno ou curl.
- A documentação de cada rota também está no campo **Documentation** da respectiva rota no Mockoon (ícone de informação ao lado do endpoint).

Documentação das APIs disponíveis com o Mock

Ambiente Mock Meli API Mock (porta 3001).
Base URL: `http://localhost:3001`

Resumo das rotas

Método	Endpoint	Autenticação	Observação
GET	`/traymeli/sellers`	Não	Sempre 404

GET	`/traymeli/sellers/252254392`	Não	Resposta aleatória 200/429
GET	`/traymeli/sellers/:id`	Não	404 se id ≠ 252254392
GET	`/mercadolibre/sites/MLB/search`	Bearer token	`seller_id` obrigatório; offset 0–25
GET	`/mercadolibre/items/:id`	Bearer token	IDs 001–030; demais 404

1. Sellers (Tray Meli)

Esta API é uma simulação do serviço de autenticação de controle de tokens da TrayMeli.

1.1 Listar sellers (sem ID)

```
None
GET /traymeli/sellers
```

Respostas

Status	Descrição
404	Bad request

1.2 Seller por ID fixo (252254392)

```
None
GET /traymeli/sellers/252254392
```

Retorno aleatório entre as respostas abaixo.

Respostas

Status	Descrição	Body
200	Token ativo	`store_id`, `user_id`, `access_token`, `inactive_token: 0`
200	Token inativo	`inactive_token: 1`
429	Too many requests	`{"status": 429, "message": "too many requests"}`

Exemplo 200 (token ativo)

```
None
{
  "store_id": "5882832",
  "user_id": 2522543922,
  "access_token":
"APP_USR-022012-62d26adf50fa550ebe1a578f66c3d53a-252254392",
  "inactive_token": 0
}
```

1.3 Seller por ID genérico

```
None
GET /traymeli/sellers/:id
```

Regras

- ID ≠ 252254392 → 404 Not Found
- ID = 252254392 → tratado pela rota 1.2 (path exato tem prioridade)

Exemplo 404

```
None
{
  "status": 404,
  "message": "Not Found"
}
```

---

2. Search Items (Mercado Livre)

Estas APIs são uma simulação das APIs do próprio Mercado Livre. As APIs originais requerem tokens válidos.

2.1 Busca de anúncios por seller

```
None
GET
/mercadolibre/sites/MLB/search?seller_id=252254392&offset=0&limit=5
```

Query params

Parâmetro	Obrigatório	Descrição
`seller_id`	Sim	Deve ser `252254392`
`offset`	Não	Default `0`. Suportado 0–25.

Respostas

Status	Condição
400	`seller_id` ausente
200	Paginação (total 30 itens)

Exemplo 200

```
None
{
  "site_id": "MLB",
  "seller_id": "252254392",
  "query": null,
  "paging": { "total": 30, "offset": 0, "limit": 5 },
  "results": ["MLB009000001", "MLB009000002", "MLB009000003",
"MLB009000004", "MLB009000005"]
}
```

Paginação

- Total de itens: 30 (IDs `MLB009000001` a `MLB009000030`).
- Use `offset` de 0 a 25 (ex.: offset=0 → 001–005, offset=5 → 006–010, …, offset=25 → 026–030).

---

3. Items (Mercado Livre)

3.1 Detalhe de item por ID

```
None
GET /mercadolibre/items/:id
Authorization: Bearer
APP_USR-022012-62d26adf50fa550ebe1a578f66c3d53a-252254392
```

Autenticação

- Header obrigatório: `Authorization: Bearer <token>`.

- Token válido.
- Se o header estiver ausente ou o token for diferente → 401 UNAUTHORIZED.

IDs válidos

- Apenas os retornados pela Search: MLB009000001 a MLB009000030.
- Qualquer outro ID → 404 com mensagem de item não encontrado.

Respostas

Status	Condição
401	Header ausente ou token inválido
404	ID fora do intervalo 001–030
200	Item encontrado

Exemplo 401

```
None
{
  "message": "UNAUTHORIZED",
  "error": "unauthorized",
  "status": 401
}
```

Exemplo 404

```
None
{
  "message": "Item with id MLB009000099 not found",
  "error": "not_found",
  "status": 404,
  "cause": []
}
```

Exemplo 200

```
None
{
  "id": "MLB009000001",
  "title": "Notebook Dell Inspiron 15 3000 Intel Core i5 8GB",
  "status": "active",
  "created": "2026-01-02T08:15:00-03:00",
  "updated": "2026-01-18T14:20:00-03:00"
}
```

- `title`: títulos fixos por ID (produtos de informática, até 60 caracteres).
- `status`: `active` ou `paused`.
- `created` / `updated`: datas em janeiro/2026, timezone -03:00.